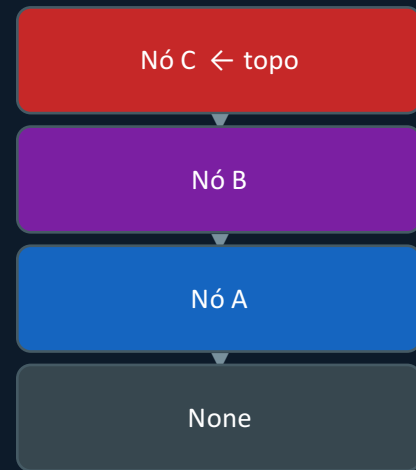


Pilhas & Filas

Estruturas de Dados com Restrição de Acesso

LIFO · FIFO · Stack · Queue · $O(1)$



A Virada de Chave

Listas

(o que você já conhece)

- Inserção em qualquer posição
- Remoção em qualquer posição
- Acesso ao meio da estrutura
- Liberdade total de acesso



mas...

Pilhas & Filas

(restrição = previsibilidade)

- Mesma base: nós encadeados
- Acesso restrito às extremidades
- Diferença: regra de negócio
- **Custo garantido: $O(1)$ estrito**

Não é nova forma de alocar memória — é uma nova regra de onde você pode mexer.

Pilha — Stack

LIFO

Last In, First Out

Metáfora:

Uma pilha de pratos — você coloca e retira somente pelo topo. Tentar puxar do meio quebra a regra da estrutura.

Ponteiro de controle: `self.topo` (apenas um)

`self.topo` →

`push()` ↓

Nó C ← `push / pop` aqui

← `pop()`

Nó B

Nó A

None

Operações da Pilha

push(valor)

$O(1)$

Insere um novo nó acima do topo atual.

```
novo.proximo = self.topo  
self.topo = novo
```

pop()

$O(1)$

Remove e retorna o nó que está no topo.

```
dado = self.topo.dado  
self.topo =  
    self.topo.proximo  
return dado
```

peek()

$O(1)$

Espia o dado do topo sem remover nada.

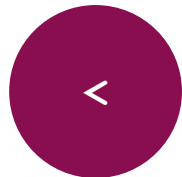
```
return self.topo.dado
```

Pilha no Mundo Real



Ctrl + Z — Desfazer

Editores empilham cada ação. Ctrl+Z faz pop e desfaz a última.



Botao Voltar do Navegador

Cada pagina visitada e empilhada. Voltar faz pop e exibe a anterior.



Call Stack — Recursao

Cada chamada de funcao empilha um frame. O retorno faz pop. E assim funciona a recursao.



Balanceamento de Parenteses

Compiladores usam pilha para verificar se cada (tem seu) e cada { tem seu }.

Fila — Queue

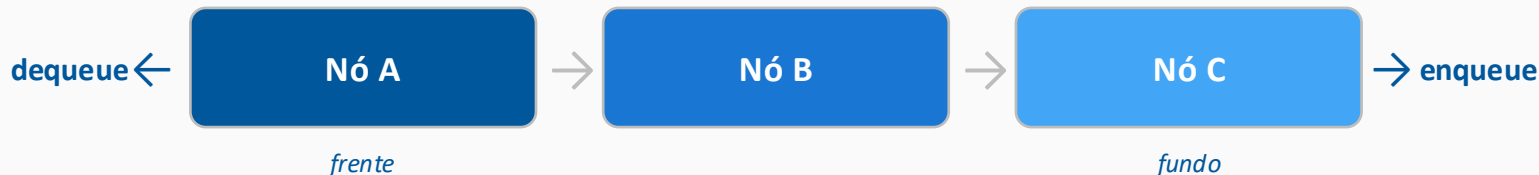
FIFO

First In, First Out

Metáfora:

Fila de banco — quem chegou primeiro e atendido primeiro.
Furar a fila ou sair do meio não é permitido.

Ponteiros: `self.frente` (saída) e `self.fundo` (entrada)



Operações da Fila

enqueue (valor)

$O(1)$

Novo nó chega ao fundo. O antigo fundo passa a apontar para ele.

```
self.fundo.proximo = novo
self.fundo = novo
```

dequeue ()

$O(1)$

Remove e retorna o nó da frente. A cabeça avança para o próximo.

```
dado = self.frente.dado
self.frente =
    self.frente.proximo
return dado
```

front ()

$O(1)$

Consulta quem é o próximo sem retirá-lo da fila.

```
return self.frente.dado
```

Fila no Mundo Real



Spooler de Impressao

Documentos sao impressos na ordem exata em que foram enviados para a impressora.



Filas de Mensagens

RabbitMQ, Kafka: tarefas assincronas processadas na ordem de chegada.



Escalonamento de Processos

O SO usa filas para gerenciar quais processos recebem tempo de CPU.



Requisicoes Web

Servidores enfileiram requisicoes HTTP para processamento justo e ordenado.

Quadro Comparativo

Critério	Pilha (Stack)	Fila (Queue)
Ordem de saída	LIFO (último entra, sai primeiro)	FIFO (primeiro entra, sai primeiro)
Pontos de acesso	1 extremidade (topo)	2 extremidades opostas (frente e fundo)
Inserção	push no topo — $O(1)$	enqueue no fundo — $O(1)$
Remoção	pop do topo — $O(1)$	dequeue da frente — $O(1)$
Ideia central	Acúmulo e reversão de passos	Atendimento justo por ordem de chegada

Recapitulando

- 1 Pilha e Fila usam a mesma base — nós encadeados. A diferença é puramente a regra de acesso.
- 2 Pilha (LIFO): um único ponteiro `self.topo`. Empilha e desempilha somente pelo topo.
- 3 Fila (FIFO): dois ponteiros `self.frente` e `self.fundo`. Entra pelo fundo, sai pela frente.
- 4 Todas as operações custam $O(1)$ — custo constante, previsível e garantido.